

Can a computer predict a random keypress?

Programming and Tools for AI – Week 2

James McDermott



Part 1: Aaronson's Oracle

AI supremacy

In the 19th century we created machines better than humans at numerical calculations

In the 20th century we created machines better than humans at many other narrow “intelligence-like” problems

In 1994 Deep Blue defeated Garry Kasparov in chess

In 2022-2026 we created machines better than good humans at writing and coding

Are you ready to experience another defeat to a machine?



Aaronson's Oracle: a simple game

The computer guesses whether you will type f or d (but doesn't tell you its guess yet).

You, the human, type f or d. You have to type fairly fast and repeat for hundreds of iterations.

If the computer does better than chance (say, 55% or more), it wins.

This game is due to Scott Aaronson, see <https://github.com/elsehow/aaronson-oracle> and <https://scottaaronson.blog/?p=2756>.

Try running it

First let's play against the finished version:

```
python W02/E01_oracle.py
```

Type f and d as unpredictably as you can, at least a hundred times. Press q to stop.

Do you still have free will?

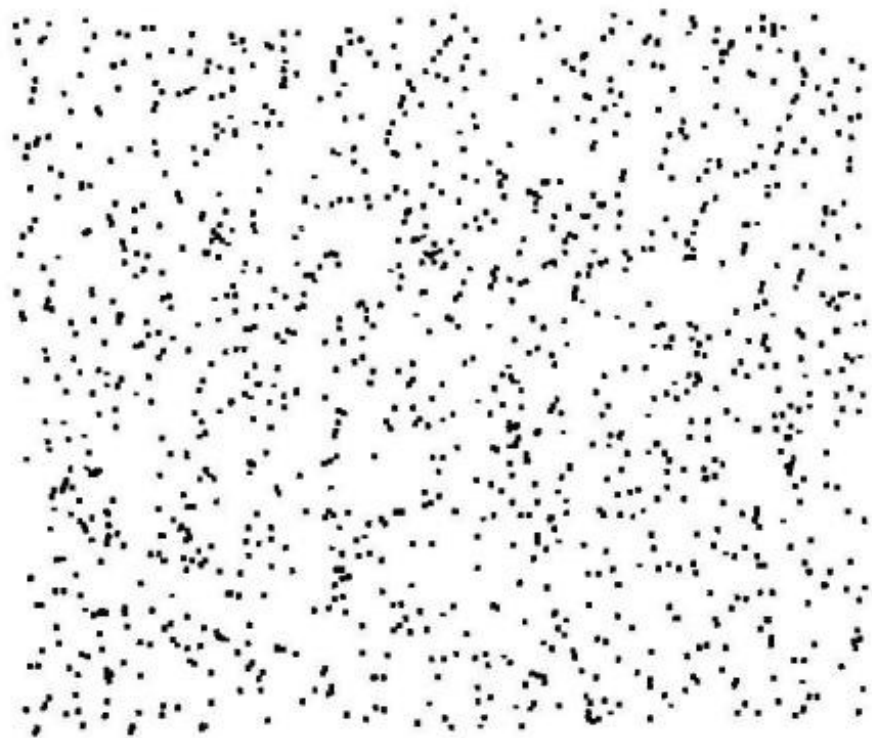
Why humans lose

To prevent prediction we need to produce randomness, ie avoid pattern. That's more difficult than it sounds!

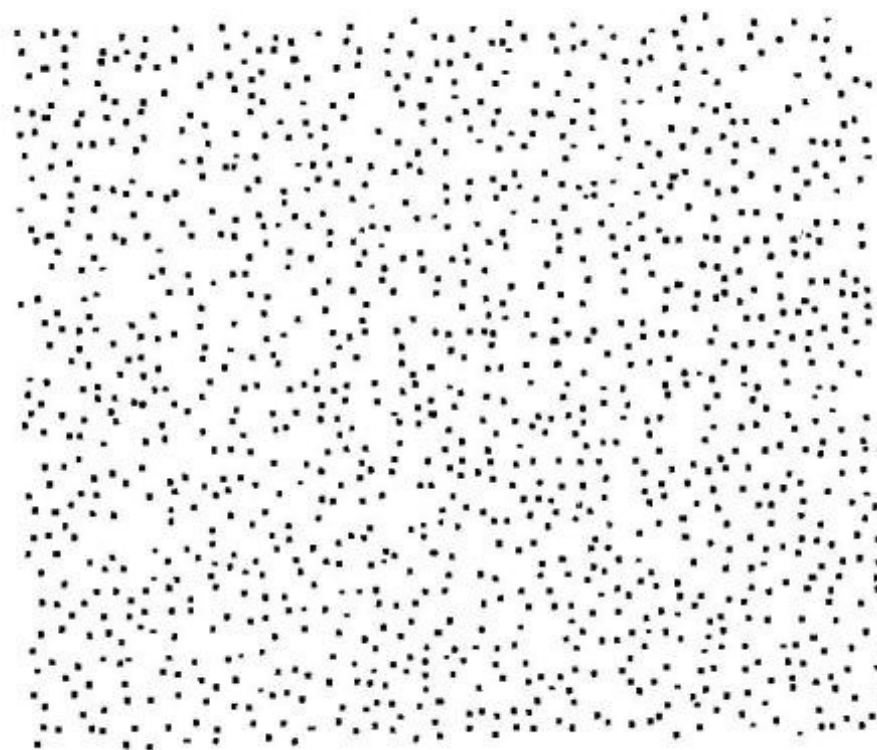
- Humans tend to avoid long repetitions: f f f f feels wrong, though in a random sequence it would happen approx 1 in 16.
- We fall into rhythms, especially typing fast: eg fd fd fd.

This leaves a trace that is not obvious to the eye, but can be detected by the right methods.

Which of these is random?

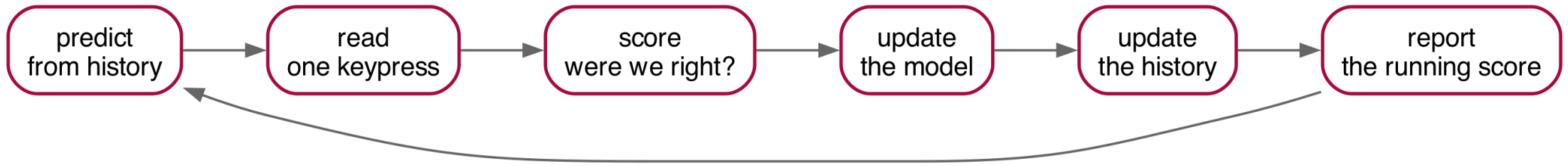


A



B

The shape of the program



Every round does the same six things, in this order. Two orderings matter: we must record our guess **before** seeing the key, and we must update the model **before** the history grows, because update uses the history as the context the key followed.

Reading one keypress

`input()` waits for you to press Return. That would make the game slow.

Doing better is fiddly, and completely different on Windows and on Mac/Linux, so it is provided in `oracle_keys.py`:

```
from oracle_keys import read_key
```

```
key = read_key()           # returns as soon as you press a key
```

That code is not examinable.

Strings, one character at a time

The history of what you typed is just a string, grown one key at a time.

```
>>> history = ""
>>> history = history + "f"
>>> history = history + "d"
>>> history
'fd'
>>> len(history)
2
```

+ on strings joins them. A string cannot be changed in place, so `history + "f"` builds a **new** string.

The game loop

```
def play():  
    history = ""  
    correct = 0  
    while True:  
        guess = predict(history)  
        key = read_key()  
        if key not in KEYS: # KEYS is just 'fd'  
            break # end the game  
        if guess == key:  
            correct += 1  
        history = history + key  
        report(guess, key, correct, len(history))
```

The pieces

- `correct += 1` is short for `correct = correct + 1`.
- `in` on a string asks about characters or substrings, whereas `in` on a list asks about items.
- `history` and `correct` are the **state**: everything the loop remembers between rounds.

Notice `play` does no predicting of its own. It calls `predict`.

Keeping score

```
def report(guess, key, correct, n):  
    print(f"guessed {guess}, you typed {key}: {correct}/{n} =  
{correct / n:.0%}")
```

guessed f, you typed d: 13/29 = 45%

:.0% inside an f-string means “as a percentage, no decimal places”. The format spec goes after a colon, inside the braces.

The stupidest possible model

```
import random
```

```
KEYS = "fd"
```

```
def predict(history):  
    return random.choice(KEYS)
```

`random.choice` picks one item from its input, each equally likely.

This ignores `history` completely. It is deliberately stupid, so we have a baseline.

Running a file, versus importing it

```
if __name__ == "__main__":  
    play()
```

We want `E01_oracle.py` to be usable in two ways:

- **Run as a script**, Python sets `__name__` to `"__main__"`, so calls `play()`.
- **Imported**, `__name__` is `"E01_oracle"`, so does not call `play()`.

This usage of `if __name__ == "__main__"` is very common.

The result

Try the stupidest possible model:

```
python W02/E01_oracle.py --no-information
```

It will get around 50%, so the human wins.

In the next part we'll make a better model, ie better predict.

Pause

We have a working game and a scoreboard. The rest of this lecture changes only the `predict` function.

Part 2: A better guess

Idea: nobody types three in a row

Here is a guess about human behaviour: after typing `f f`, it would be “too predictable”, to type another `f`, so you switch to `d`.

- See `f f`, predict `d`.
- See `dd`, predict `f`.
- Otherwise, no opinion — guess at random as before.

Implementation

```
def predict(history):  
    if history[-2:] == "ff":  
        return "d"  
    elif history[-2:] == "dd":  
        return "f"  
    else:  
        return random.choice(KEYS)
```

The last two characters

`history[-2:]` is **slicing**, to get the final two characters.

```
>>> "fdfdff"[-2:] # final two characters  
'ff'
```

```
>>> "f"[-2:] # no error, just short output  
'f'
```

```
>>> ""[-2:] # no error, just empty output  
''
```

Note the last two: asking for more characters than exist is **not** an error. So the first two rounds need no special case: "" and "f" are equal to neither "ff" nor "dd", and we guess randomly.

Aside: **slicing** is “forgiving”, ie throws no error, whereas **indexing** (`history[-2]`) would raise `IndexError`.

Who wins with this model?

```
python W02/E01_oracle.py --no-3-repeats
```

It's too hard to introspect and write down the patterns

I (as a human) was able to beat the no-3-repeats model by avoiding its expectations. But probably more complex patterns still exist in my typing. To create a stronger program I would have to **hand-code** them, but I don't know what they are!

But the history contains useful information. We should count what actually happens, rather than code up what we think happens.

In other words, let's use **learning**.

Part 3: Learning

Stop guessing the rule, instead count

Instead of asking “what do people do after f f?”, ask the data:

In this history, every time f f appeared, what came next?

Then predict whichever came next more often.

The same machinery works for f f, for f d f d, for any short run of recent keys. We do not have to know which patterns matter, because we count them all.

n-grams

An **n-gram** is a run of n consecutive symbols.

"fdffd" contains these 3-grams:

fdf dff ffd

We can see an n-gram as (n-1 characters of context, 1 character of outcome).

Eg, we see fdf as "the context is fd, followed by f".

Counting them in a dict

A dict maps **keys** to **values**. Here, an n-gram maps to a count.

```
>>> counts = {}
>>> counts["fdf"] = 4
>>> counts["fdd"] = 1
>>> counts["fdf"]
4
>>> "fdf" in counts
True
>>> counts
{'fdf': 4, 'fdd': 1}
```

A list looks things up by position, whereas a dict looks them up by name. (Actually, dict can also store and look up in other ways, but we will see that later.)

dict: the pieces

- `{}` is an empty dict. `{"a": 1, "b": 2}` is a literal.
- `d[k]` reads, `d[k] = v` writes, creating the key if it is not there.
- `k in d` asks about **keys**, never values.
- `len(d)` is the number of pairs.
- `for k in d:` iterates the keys. `for k, v in d.items():` iterates both.
- Keys must be unique.

The missing key problem

```
>>> counts = {}  
>>> counts["fdf"] # crash  
KeyError: 'fdf'
```

An n-gram we have never seen is not in the dict, and asking for it is an error. But “never seen” should mean **zero**, not a crash.

```
>>> counts.get("fdf", 0) # 0 is the default  
0
```

`d.get(k, default)` is a no-crash version of `d[k]`.

Counter: a dict that starts at zero

```
>>> from collections import Counter
>>> counts = Counter()
>>> counts["fdf"] += 1
>>> counts["fdf"] += 1
>>> counts["ddd"]
0
>>> counts
Counter({'fdf': 2})
```

A Counter **is** a dict, with one difference: a missing key reads as 0 instead of raising an error. So `counts[ngram] += 1` always works.

Note `counts["ddd"]` returns 0, but doesn't create a new key ddd.

The collections module

<code>Counter()</code>	a dict where a missing key reads as 0
<code>Counter(xs)</code>	counts how often each item of xs appears
<code>defaultdict(int)</code>	like <code>Counter()</code> , the 0 is created by calling <code>int()</code>
<code>defaultdict(list)</code>	a dict where a missing key starts as <code>list()</code>

`defaultdict(list)` is for collecting rather than counting.

Predicting from the counts

`K = 4` *# context length: K + 1 is the n of "n-gram"*

```
def predict(counts, history):  
    context = history[-K:]  
    f = counts[context + "f"]  
    d = counts[context + "d"]  
    if f > d:  
        return "f"  
    elif d > f:  
        return "d"  
    else: # if they are equal  
        return random.choice(KEYS)
```


Learning from the counts

A very simple learning algorithm: literally just count.

```
def update(counts, history, key):  
    """Record that `key` followed the end of `history`."""  
  
    >>> counts = Counter()  
    >>> update(counts, "fdfd", "f")  
    >>> update(counts, "fdfd", "f")  
    >>> update(counts, "fdfd", "d")  
    >>> counts["fdfdf"]  
    2  
    >>> counts["fdfdd"]  
    1  
    """"  
    counts[history[-K:] + key] += 1
```

Why the model is a pair of functions

`predict` and `update` together are an ML **model**.

- Swap `predict` for the random one, or the `ff/dd` one: the rest of the game still works. That is all `--no-information` and `--no-3-repeats` do.
- The model is not directly attached to the keyboard, so we could run it over a saved file of keypresses.
- We can doctest a model, but cannot doctest via the keyboard. That is why `update` above has tests and `play` does not.

Where the state lives

counts is the model **state**.

```
def play():  
    counts = Counter()  
    history = ""  
    while True:  
        guess = predict(counts, history)  
        ...  
        update(counts, history, key)
```

Part 4: How good is it?

Does it work?

Try it on yourself, a few hundred keypresses each:

```
python W02/E01_oracle.py --n-gram 2
```

```
python W02/E01_oracle.py --n-gram 5
```

```
python W02/E01_oracle.py --n-gram 12
```

Did you beat the machine (less than about 55%)? What value of n was best?

Best values of n

	200 keys	500 keys	3000 keys
n=2	80%	80%	82%
n=3	78%	80%	82%
n=4	81%	82%	85%
n=5	78%	80%	85%
n=6	79%	79%	85%
n=8	74%	74%	84%
n=12	64%	66%	78%

Small n: the context is too simple, eg predicting based on just 1 or 2 characters.

Large n: we would need millions of keypresses to get reliable counts for every context.

More data enables larger n and better accuracy.

Aside, not examinable: this is the **bias–variance trade-off**, an essential concept in ML.

Accuracy against a **simulated** typist. Did yours show the same trends?

What did it learn about you?

```
>>> counts.most_common(5)
[('fdfdf', 89), ('dfdfd', 81), ('fdffd', 18), ('dffdf', 18),
 ('dfdff', 17)]
```

`Counter.most_common(n)` gives the `n` commonest, largest first.

All of these are 5-tuples which **should** all (if we were typing randomly) be equally common!

Is this AI? It is definitely ML.

It is easier to learn the user's patterns than to write them down by hand. Even the user does not *know* their own patterns — but the patterns *do* exist. So this is a perfect opportunity for ML.

In the standard vocabulary:

- K is a model **hyperparameter** which controls model configuration. It is chosen by us, based on testing and theory.
- The n-gram counts are the model **parameters**, learned from data.
- It is an **online** model: it predicts from the first keypress, and learns from every one.

Aside, not examinable: this type of sequential online prediction problem is called **prequential**.

Another link to AI

“The amount of information” in an object is a key concept in several areas of AI. Eg:

s1 = 'aaaaaaaaaaaaaaaaaaaaaaaaa'

s2 = 'abcdabcdabcdabcdabcd'

s3 = 'abaabddbcbabdbabcccadacc'

s1 contains the least information, s2 is in the middle, and s3 contains the most. We could use **the number of distinct n-grams** as a crude measure of the amount of information in each string.

LZW is one example of a more sophisticated approach.

Another link to AI

n-grams can't capture everything. Eg:

```
s4 = '112123123412345123456'
```

Here n-grams would fail to capture the regularity that you can easily see.

Kolmogorov complexity of an object is “the length of the shortest program that would output the object”.

```
s1 = 'aaaaaaaaaaaaaaaaaaaaaaaaaaaa' # 'a' * 24
```

```
s2 = 'abcdabcdabcdabcdabcdabcd' # 'abcd' * 6
```

```
s3 = 'abaabddbcbbabdbabcccadacc' # no short program
```

```
s4 = '112123123412345123456'      # for fun: write one
```

We won't pursue this further.

Part 5: Saving what we learned

The model forgets

When we close the program, the Counter is gone. Next time it has to learn from scratch.

We want to **save** the counts to a file so we can **load** them back.

The counts are the model. A saved model is a file.

Choose a format

We will write a plain text file containing one n-gram per line, then a space, then the count:

ddffd 7

ddfdf 12

dfdfd 29

Writing a file

```
def save(counts, path):  
    with open(path, "w") as f:  
        for ngram in sorted(counts):  
            f.write(f"{ngram} {counts[ngram]}\n")
```

- `open(path, "w")` opens for **writing**, and empties any existing file. "r" reads (the default), "a" appends.
- `f.write` does not add a newline. `print` does. Hence the `\n`.
- `sorted(counts)` walks the keys in alphabetical order (not most-common order), so the file has the same order between runs, and is easy to diff.

with

```
with open(path, "w") as f:  
    ...
```

`with` closes the file when the block ends — whether it ends normally or by raising an exception.

Without it, an error halfway through would leave the file open, and what you wrote may still be sitting in a buffer, not on the disk. You will often see plain `f = open(...)`, but using `with` is better practice.

Reading it back

```
def load(path):  
    counts = Counter()  
    with open(path) as f:  
        for line in f:  
            ngram, n = line.split()  
            counts[ngram] = int(n)  
    return counts
```

- Iterating a file for `line in f` gives one **line** at a time, including final `\n`.
- `line.split()` splits on whitespace and drops the `\n`, giving `['dfdfd', '29']`.
- We use **unpacking** to separate that list into two variables `ngram, n`
- Both are strings, so we convert `n` with `int(n)`. Eg `"29" + 1` would be an error.

Round trip

A good test for any save/load pair is to round-trip:

```
>>> save(counts, "ngrams.txt")  
>>> load("ngrams.txt") == counts  
True
```

Where files live (not examinable)

```
from pathlib import Path
```

```
DATA = Path(__file__).parent / "data"  
counts = load(DATA / "ngrams.txt")
```

- A path like "data/ngrams.txt" is relative to **wherever you ran python from**, not relative to the .py file.
- `__file__` is the path of the current .py file.
- `/` joins subdirectories with platform-specific separator (`/` on Mac/Linux, `\` on Win).

Summary

- A game loop that predicts, reads, scores and updates.
- Three models using the same interface.
- Machine learning! The ML model is stored in a Counter which is a type of dict.
- A plain-text file to save the model.

New this week: dict, Counter, d.get, open/with, f.read, f.write, line.split, int, if __name__ == "__main__".